

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

MASTER IN ICT FOR INTERNET AND MULTIMEDIA

Final Project Report of The Course CyberPhysical Systems and IoT Security

Title of the reference paper:

Authentication of IoT Device and IoT Server Using Secure Vaults

Group Members

Surname	Name	Student ID
NEMMICHE	Mohamed Mounib	2184905
SAIDI	Hassiba Lina	2142744
PERIN	Alessandro	2203653

ACADEMIC YEAR
2025/2026

1 Objectives

The main objective of this project is to **implement and evaluate a lightweight mutual authentication protocol for IoT devices** based on the concept of *Secure Vaults*, as described in the reference paper “*Authentication of IoT Device and IoT Server Using Secure Vaults*” by S. Srinivas, A. Kumar et al. [1]. The protocol enables both the IoT device and server to mutually verify each other’s identity using only symmetric-key primitives (AES, XOR, HMAC), making it suitable for resource-constrained IoT environments.

More specifically, the project aims to:

- Design and implement a **Python-based simulation** of the full 5-step Secure Vault mutual authentication handshake, including vault initialization, challenge-response exchange, session key derivation, and post-session vault update for forward secrecy.
- Compare the **performance** of the Secure Vault protocol against two alternative authentication methods, **ECC-based authentication (P-256 ECDSA)** and **Simple Password (SHA-256 challenge-response)**, using execution time (ms) as a proxy for energy consumption on resource-constrained devices.
- Evaluate **scalability** by measuring per-device authentication time across varying numbers of devices (10, 50, 100, 500, 1000), with multiple independent runs for statistical robustness.
- Validate the **security properties** of the protocol by simulating four classes of attacks: **Brute Force Key Guessing**, **Vault Key Recovery via GF(2) Linear Algebra**, **Replay Attacks**, and **Man-in-the-Middle (MITM) Attacks** with three distinct strategies.

The project is divided into **two parts**:

1. **Part 1 – Protocol Implementation & Performance Experiments**: Full implementation of the Secure Vault protocol and comparative benchmarking against ECC and Simple Password authentication.
2. **Part 2 – Security Attack Simulations**: Systematic validation of the protocol’s resilience against cryptographic and protocol-level attacks.

2 System Setup & Implementation

This section describes the architecture of the simulation framework, the main Python modules, and how they interact to model the Secure Vault authentication protocol. The implementation is entirely software-based, using Python 3 and standard cryptographic libraries.

2.1 Repository Structure

To maintain a clean and scalable architecture, the Python simulation is modularized into several core components. The global configuration (such as vault size N , key size M , and challenge size P) is centralized in `config.py`, while `main.py` serves as the primary entry point. The core protocol logic resides entirely within the `entities/` directory; this includes the low-level cryptographic primitives (`crypto_utils.py`) alongside the object-oriented entity classes representing the device (`device.py`), the server (`server.py`), and the shared key storage (`vault.py`).

The evaluation scripts are separated by their objective. The `experiments/` directory contains the performance benchmarking framework used to compare the Secure Vault against ECC and Simple Password methods. Conversely, the `Part 2-Attacks/` directory houses the scripts used to simulate the four security threat models (brute force, GF(2) key recovery, replay, and MITM attacks). All generated charts and execution logs are automatically saved to the `results/` folder. The complete source code is publicly available in the GitHub repository referenced at the end of this report.

2.2 Protocol Overview

The Secure Vault protocol implements a **5-step mutual authentication handshake** between an IoT device and a server. Both parties share an identical vault, a set of N symmetric keys, established during an initial secure provisioning phase. The protocol flow is illustrated in Figure 1.

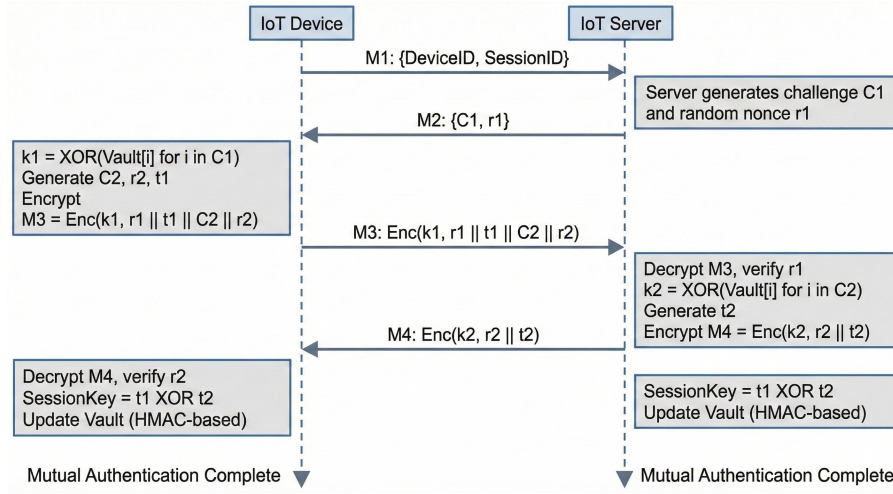


Figure 1: The Secure Vault protocol follows a 4-message mutual authentication handshake.

The steps are:

1. **Step 1 (Device → Server):** The device sends message M1 containing its `DeviceID` and a random `SessionID`.
2. **Step 2 (Server → Device):** The server generates a challenge C1 (P random vault indices) and a nonce $r1$. It derives key $k1 = \text{XOR}(\text{Vault}[i] \text{ for } i \text{ in } C1)$ and sends $M2 = \{C1, r1\}$.
3. **Step 3 (Device → Server):** The device derives the same $k1$ from its vault copy, generates its own challenge $C2$, nonce $r2$, and a temporary key $t1$. It sends $M3 = \text{Enc}(k1, r1 || t1 || C2 || r2)$.
4. **Step 4 (Server → Device):** The server decrypts $M3$ with $k1$, verifies $r1$ (authenticating the device), derives $k2 = \text{XOR}(\text{Vault}[i] \text{ for } i \text{ in } C2)$, generates $t2$, and sends $M4 = \text{Enc}(k2, r2 || t2)$.
5. **Step 5 (Finalization):** The device decrypts $M4$ with $k2$, verifies $r2$ (authenticating the server). Both parties compute the session key as $\text{SessionKey} = t1 \text{ XOR } t2$ and update their vaults using this session key.

2.3 Cryptographic Primitives

The `crypto_utils.py` module provides the following operations:

- `generate_random_key(size)`: Uses `os.urandom()` (CSPRNG) to generate cryptographically secure random keys.
- `xor_bytes(a, b)`: Element-wise XOR of two equal-length byte strings, used for key derivation and session key computation.
- `xor_multiple_keys(keys)`: Chains XOR across a list of keys, the core of challenge-response key derivation.
- `aes_ecb_encrypt` / `aes_ecb_decrypt`: AES-128-ECB with PKCS7 padding, used to encrypt protocol messages M3 and M4.

- `compute_hmac(key, message)`: HMAC-SHA256 for message integrity.
- `hmac_expand(key, data, length)`: Iterative HMAC-based key expansion (HKDF-like), used for vault updates to achieve forward secrecy.

2.4 Secure Vault

The `SecureVault` class (in `vault.py`) is the core data structure:

- **Initialization**: Generates N random keys, each of M bytes (default: $16 \text{ keys} \times 16 \text{ bytes} = \text{AES-128}$).
- **Challenge Generation**: `generate_challenge(p)` selects P random unique indices from $[0, N-1]$. The challenge space is $C(N, P) = C(16, 6) = \mathbf{8,008 \text{ possible challenges}}$.
- **Key Derivation**: `derive_key_from_indices(indices)` computes $\text{XOR}(\text{Key}[i] \text{ for } i \text{ in indices})$, producing the shared secret for encrypting protocol messages.
- **Vault Update (Forward Secrecy)**: After each successful session, the vault is updated:
 1. Concatenate all vault keys: `vault_concat = Key[0] || Key[1] || ... || Key[N-1]`
 2. Expand via HMAC: `expanded = HMAC_expand(session_key, vault_concat, N * M bytes)`
 3. XOR each key: `New_Key[i] = Old_Key[i] XOR expanded[i*M : (i+1)*M]`

This ensures that **old session keys cannot decrypt future messages** and **compromising a current vault does not reveal past sessions**.

2.5 IoT Device

The `IoTDevice` class (in `device.py`) implements the client-side protocol logic:

- `step1_initiate()`: Generates a random `SessionID` and sends $M1 = \{\text{DeviceID}, \text{SessionID}\}$.
- `step3_respond(m2)`: Receives $C1$ and $r1$, derives $k1$ from the vault, generates $C2$, $r2$, and $t1$, then encrypts and sends $M3$.
- `step5_finalize(m4)`: Decrypts $M4$ with $k2$, verifies $r2$ (server authentication), computes the session key, and updates the vault.

2.6 IoT Server

The `IoTServer` class (in `server.py`) manages multiple registered devices, each with its own vault copy:

- `register_device(device_id, vault)`: Stores a clone of the device's vault.
- `step2_challenge(m1)`: Validates the device ID, generates $C1$ and $r1$, derives $k1$, and sends $M2$.
- `step4_verify_and_respond(device_id, m3)`: Decrypts $M3$, verifies $r1$ (device authentication), derives $k2$, generates $t2$, sends $M4$, computes session key, and updates vault.

2.7 Experiment Comparison Framework

The `experiments.comparison.py` module benchmarks three authentication methods:

Each method is benchmarked using `time.perf_counter()` with high-precision timing across multiple runs.

Method	Approach	Security Properties
Secure Vault	XOR-based key derivation + AES-128 + HMAC vault update	Mutual authentication + forward secrecy
ECC (P-256 ECDSA)	Elliptic curve key generation + digital signatures	Mutual authentication, no forward secrecy
Simple Password	SHA-256 challenge-response (nonce-based)	One-way authentication only, no forward secrecy

Table 1: Comparison of Authentication Methods

3 Experiments

This section details the experimental methodology used to validate the Secure Vault protocol implementation and compare its performance against alternative authentication methods.

3.1 Experimental Methodology

The experiments were structured into two categories:

1. **Protocol Correctness Validation:** A demo of 3 consecutive authentication sessions verifying that mutual authentication succeeds, session keys match on both sides, and vaults remain synchronized after updates.
2. **Performance Comparison:** Benchmarking the Secure Vault protocol against ECC (P-256) and Simple Password authentication across varying device counts.

3.2 Simulation Parameters

All experiments used the following configuration (from `config.py`):

- **Vault Size (N):** 16 keys (reduced from the paper’s $N=128$ for simulation speed while maintaining adequate entropy)
- **Key Size (M):** 16 bytes (AES-128 standard)
- **Challenge Size (P):** 6 keys per challenge (challenge space = $C(16, 6) = 8,008$)
- **Device Counts:** 10, 50, 100, 500, 1000
- **Runs per Datapoint:** 5 (for statistical averaging)
- **ECC Curve:** NIST P-256
- **Password Length:** 16 bytes

3.3 Data Collection

Data was collected using a custom `TeeWriter` class that simultaneously writes to the terminal and to timestamped `.log` files. Two distinct log files were generated:

- `demo_and_experiments_20260220_081203.log`: Contains the full protocol demo trace (3 sessions with hex values for all messages) and the complete experiment results table with mean, standard deviation, min, and max timing for each method and device count.
- `security_attacks_20260220_090134.log`: Contains the results of all four attack simulations with detailed outcomes.

Three performance plots were generated:

- `bar_chart_comparison.png`: Grouped bar chart comparing per-device authentication time across methods and device counts.
- `line_chart_scaling.png`: Line chart showing authentication time scaling with device count.
- `bar_chart_log_scale.png`: Logarithmic-scale bar chart (necessary because ECC is an order of magnitude slower).

4 Results and Discussion

This section presents the results obtained from both the protocol demo and the performance experiments.

4.1 Protocol Correctness

The protocol demo validated the complete 5-step handshake across 3 consecutive sessions. Below is a condensed trace from Session 1 (from `demo_and_experiments_20260220_081203.log`):

```
[Step 1] Device -> Server: M1
Device ID: Device_001 | Session ID: 4b64d1...

[Step 2] Server -> Device: M2
Challenge C1: [14, 10, 8, 3, 13, 6] | Nonce r1: f37e00...

[Step 3] Device -> Server: M3
Ciphertext: 8e361b... (encrypted with k1 derived from C1)

[Step 4] Server -> Device: M4
Ciphertext: 277dae...
Server verified r1 - device is authenticated!

[Step 5] Finalization
Device verified r2 - server is authenticated!
Session Key (device & server): 1b4fa1... (Keys match: True)
Vault updated for next session.
```

All 3 sessions completed successfully, confirming:

- **Mutual authentication:** Both device and server verified each other's identity via nonce validation.
- **Session key agreement:** Both parties derived identical session keys.
- **Vault synchronization:** Vaults remained in sync after HMAC-based updates, enabling seamless multi-session operation.

4.2 Performance Comparison

The core experiment compared per-device authentication time (ms) across three methods and five device counts (10, 50, 100, 500, and 1000). The raw execution logs and detailed statistical breakdowns are fully visualized in the charts below.

The visual comparisons are shown in Figures [2](#), [3](#), and [4](#).

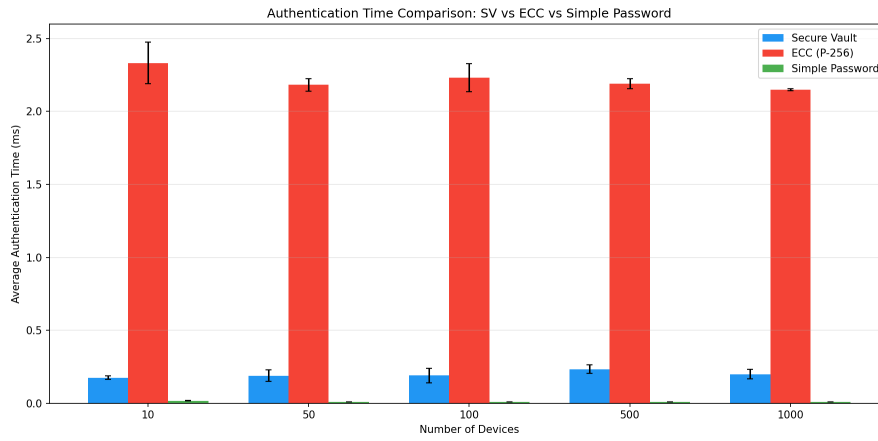


Figure 2: Per-device authentication time comparison across methods and device counts.

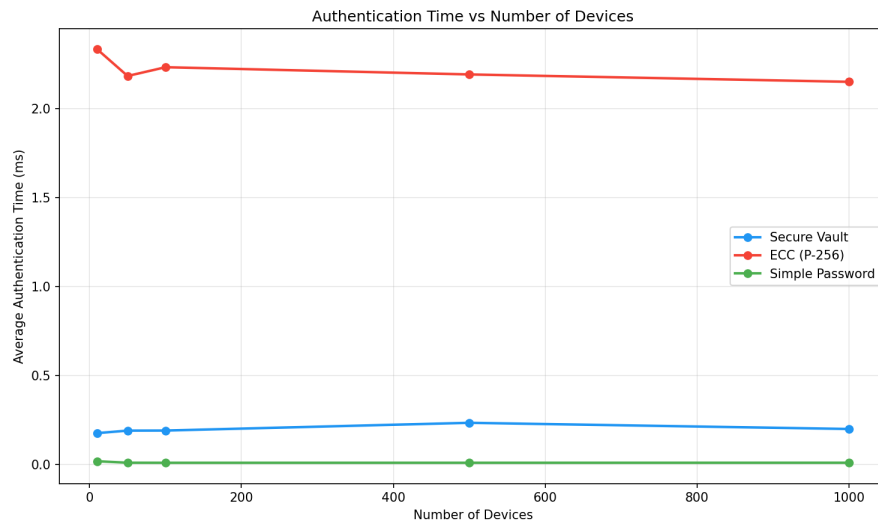


Figure 3: Authentication time scaling with number of devices.

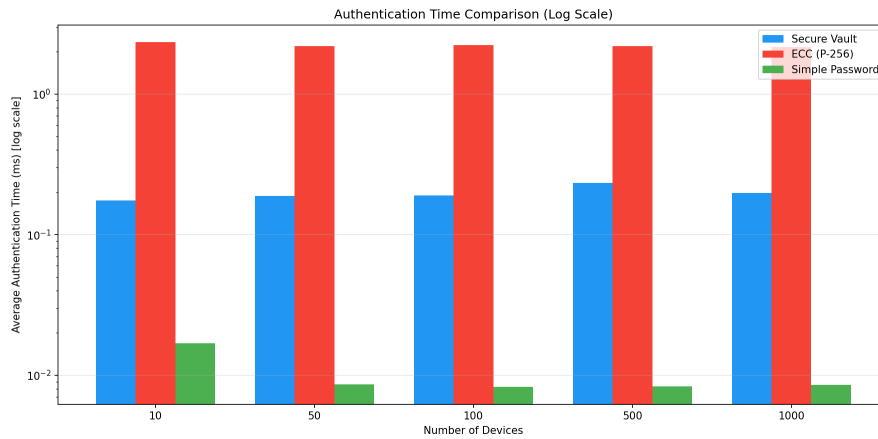


Figure 4: Log-scale comparison highlighting the order-of-magnitude difference between ECC and other methods.

Key Findings

- **ECC is approximately 12× slower than Secure Vault:** ECC authentication averages ~ 2.2 ms per device compared to ~ 0.19 ms for Secure Vault. This is due to the computationally expensive elliptic curve key generation and digital signature operations (ECDSA sign + verify), which dominate the ECC protocol’s execution time.
- **Simple Password is the fastest but least secure:** At ~ 0.008 ms per device, Simple Password is roughly $23\times$ faster than Secure Vault. However, it provides **only one-way authentication** (server verifies device, but the device cannot verify the server) and **no forward secrecy**. This makes it unsuitable for security-critical IoT deployments.
- **Secure Vault achieves the optimal security-performance balance:** It provides full mutual authentication and forward secrecy at a cost of only ~ 0.19 ms per device — fast enough for resource-constrained IoT devices while offering security guarantees comparable to or exceeding ECC.
- **Constant scalability:** Per-device authentication time remains remarkably stable across 10 to 1,000 devices for all three methods, confirming that the protocol overhead scales at $O(1)$ per device.

4.3 Limitations

While the simulation successfully implements the protocol described in the reference paper [1], a few limitations should be noted:

1. **PC Simulation vs. Real IoT Hardware:** Execution times on a desktop PC serve as a proxy for energy consumption but do not directly translate to the timing on constrained microcontrollers (e.g., ARM Cortex-M). The relative ordering (Secure Vault $\ll$ ECC) is expected to hold on real hardware since symmetric operations are significantly cheaper than asymmetric ones.
2. **Reduced Vault Size:** The vault uses $N=16$ keys instead of the paper’s suggested $N=128$ for practical simulation speed. While this reduces the challenge space from $C(128, 6)$ to $C(16, 6) = 8,008$, it maintains sufficient entropy for demonstrating the protocol mechanics.
3. **AES-ECB Mode:** The implementation uses AES-ECB rather than AES-CBC or AES-GCM. While ECB is simpler and sufficient for this protocol (where each encrypted message has unique content due to random nonces), a production deployment should consider authenticated encryption modes.

5 Part₂ : Security Attack Simulations

While Part 1 focused on the protocol’s functionality and performance, this section presents the results of four attack simulations designed to validate the protocol’s security claims. All attacks were implemented in the **Part 2-Attacks/** module.

5.1 Attack 1: Brute Force Key Guessing

Threat Model: An attacker eavesdrops on M2 (which contains the challenge C1 and nonce r1) and M3 (the encrypted response). The attacker then attempts to guess the derived key k1 by trying random 128-bit keys to decrypt M3 and recover r1.

Implementation: The simulation runs 50,000 random key guesses, attempting to decrypt M3 with each and checking if the decrypted plaintext begins with the known r1.

Result: 0 out of 50,000 attempts succeeded. The key space is 2^{128} , giving a per-attempt success probability of approximately 1.47×10^{-34} . Even with billions of attempts, brute force remains computationally infeasible.

5.2 Attack 2: Vault Key Recovery (GF(2) Linear Algebra)

Threat Model: An attacker collects multiple challenge-response pairs from observed sessions and attempts to solve for individual vault keys using constraint propagation over GF(2) (Galois Field).

Implementation: The simulation observes 50 sessions and collects the challenge indices (C1 values) along with the derived keys. It then attempts to solve the resulting system of linear equations over GF(2) to recover the N=16 individual vault keys. Two scenarios are tested:

- **Without vault rotation:** All equations reference the same static vault (best case for attacker).
- **With vault rotation:** Equations reference different vault states after each session update (realistic scenario).

Result: 0 out of 16 keys recovered in both scenarios. Without rotation, the system is underdetermined (P=6 unknowns per equation with N=16 total keys). With rotation, the equations become inconsistent because each session's vault state differs.

5.3 Attack 3: Replay Attack

Threat Model: An attacker captures M1 and M3 from a legitimate session and replays them in a later session after the vault has been updated.

Implementation: A legitimate session is completed successfully, causing both device and server vaults to update. The attacker then replays the old M1 and M3 to the server.

Result: The server **rejected** the replayed M3. Since the vault was updated after the original session, the server's k1 (derived from the new vault) differs from the k1 used to encrypt the old M3. Decryption produces garbage, and the r1 verification fails. This confirms that the **HMAC-based vault update provides effective replay protection**.

5.4 Attack 4: Man-in-the-Middle (MITM)

Threat Model: An attacker intercepts and modifies messages between the device and server. Three distinct strategies were tested:

- **Strategy A, Modify r1 in M2:** The MITM intercepts M2, replaces the nonce r1 with a different value, and forwards the modified M2 to the device. The device encrypts the tampered r1 in M3. When the server decrypts M3 and checks r1, the value does not match. **Result: BLOCKED.**
- **Strategy B, Forge M3 with random key:** The MITM crafts a fake M3 by encrypting a fabricated payload using a random key (not derived from the vault). The server decrypts with the real k1, producing garbage plaintext that fails r1 validation. **Result: BLOCKED.**
- **Strategy C, Bit-flipping on M3 ciphertext:** The MITM flips one bit in the middle of the M3 ciphertext. AES-ECB decryption corrupts the affected block, resulting in either invalid PKCS7 padding (decryption error) or a mangled r1 that fails verification. **Result: BLOCKED.**

5.5 Security Attack Summary

The complete results from `security_attacks_20260220.090134.log`:

=== Security Attack Summary ===

Attack	Type	Status
Brute Force Key Guessing	Cryptographic	SECURE
Vault Key Recovery (GF(2))	Cryptographic	SECURE
Replay Attack	Protocol	SECURE

Man-in-the-Middle (MITM)	Protocol	SECURE
+-----+-----+-----+		

All attacks FAILED to breach the protocol.

The Secure Vault protocol resisted all tested attack vectors.

The protocol's security rests on three pillars:

1. **Vault secrecy:** Without knowing the vault contents, an attacker cannot derive the encryption keys (k_1 , k_2), making brute force and algebraic recovery infeasible.
2. **Nonce-based authentication:** The random nonces r_1 and r_2 ensure that each session is unique and that both parties can verify each other's identity through correct decryption.
3. **Vault update (forward secrecy):** The HMAC-based vault rotation after each session ensures that compromising a current session does not reveal past or future sessions, and that old captured messages cannot be replayed.

6 Conclusion

In this project, we implemented and evaluated the Secure Vault mutual authentication protocol for IoT devices, as described in the reference paper [1].

Through our Python simulation, we successfully demonstrated:

- A complete **5-step mutual authentication handshake** that reliably authenticates both device and server, with consistent session key agreement and vault synchronization across multiple sessions.
- A **performance advantage over ECC-based authentication:** The Secure Vault protocol is approximately $12\times$ faster (~ 0.19 ms vs. ~ 2.2 ms per device), making it significantly more suitable for resource-constrained IoT devices that cannot afford expensive asymmetric cryptographic operations.
- **Linear scalability:** Per-device authentication time remains stable from 10 to 1,000 devices, confirming the protocol's suitability for large-scale IoT deployments.
- **Robust security:** All four attack simulations, brute force, algebraic recovery, replay, and MITM, failed to breach the protocol. The combination of vault-based key derivation, nonce verification, and HMAC-based vault rotation provides strong security guarantees including mutual authentication and forward secrecy.

The Secure Vault protocol achieves an optimal trade-off between security and performance: it offers security properties comparable to or exceeding public-key approaches while operating exclusively with lightweight symmetric primitives (AES, XOR, HMAC) that are well-suited for constrained IoT environments.

Group Members Contributions

- **Mohamed Mounib NEMMICHE:** I was responsible for the code implementation. This included developing the core Secure Vault protocol (**entities** module) and the performance experiment framework.
- **Hassiba Lina SAIDI:** I was responsible for the documentation. This involved analyzing the simulation data, synthesizing the theoretical concepts from the reference paper, and structuring the final report.
- **Alessandro PERIN:** I was responsible for reviewing the implementation and implementing the attack simulations. This included verifying the protocol correctness, cross-checking the experimental results, and providing feedback on the security analysis.

References

- [1] S. Srinivas, A. Kumar *et al.*, “Authentication of iot device and iot server using secure vaults,” *IEEE Internet of Things Journal*, 2018.
- [2] “Source code link of the final project - cyberphysical systems and iot security course,” <https://github.com/MounibNemmiche/FinalProject-CPS-IoTSec-IoTAuthentication>.